ANPR BACKEND | TECHNICAL DOCUMENTATION

Database Architecture and Schema Report

Automatic Number Plate Recognition Backend

Scope: SQLite database design, table schemas, relationships, constraints, indexes, backend integration, data integrity, and security considerations.

INPUT FILE	PURPOSE
anpr.db	Database inspected for actual tables, indexes, aggregate row counts, and integrity status.
schema.sql	DDL source reviewed to document columns, keys, constraints, and intended relationships.
Backend boundary	API/backend and database only; the computer-vision component is treated as a separate producer of plate-recognition results.

Prepared from the project files provided on 25 October 2026. No personal row values are reproduced in this report.

1. Executive Summary

The supplied SQLite database, anpr.db, contains 12 application tables and 29 application rows at the time of inspection. The model covers user accounts and vehicles, membership plans and payments, garages and gates, parking sessions, access decisions, gate commands, audit records, and notifications.

The schema is designed to connect a plate read to a registered vehicle, link the vehicle to its owner, determine whether the owner has an eligible membership, record the access decision, and maintain supporting operational records. The uploaded computer-vision component is outside this database report; it would typically supply the recognized plate string and confidence value to the backend.

> Important schema alignment note: the older example code referred to bakery_security_logs and history. Neither table exists in this uploaded schema/database. In the current model, vehicles is the registered-vehicle lookup table, access_events is the access decision/event log, and parking_sessions records the vehicle's stay in a garage.

2. Scope and Evidence

This report was produced by inspecting both the supplied schema.sql file and the actual SQLite catalog in anpr.db. The data overview uses aggregate counts only; names, emails, phone numbers, license plates, password hashes, messages, and image paths are deliberately not displayed.

The schema contains 12 application tables plus SQLite-managed metadata named sqlite_sequence. The latter is not a business entity and is excluded from the application-table count.

Inspection item	Observed result
Database file size	84 KiB (86,016 bytes) at inspection time.
Application tables	12
Application rows	29
SQLite integrity_check	ok
foreign_key_check	No violations returned by the inspection query.
Foreign-key enforcement PRAGMA	PRAGMA foreign_keys returned 0 on the standalone inspection connection. See Section 11.
Database triggers	0

The findings describe the uploaded file as it existed during inspection. Counts will change as the team adds users, vehicles, access events, and other records.

3. Application Table Inventory

Table	Main responsibility	Rows in uploaded DB
users	Accounts, roles, and account status	6
vehicles	Vehicle registration and normalized plate lookup	6
plans	Membership plan definitions and pricing	2
memberships	User subscription periods and statuses	5
payments	Payments attached to memberships	2
garages	Garage/location definition and capacity	1
gates	Entry/exit gates that belong to a garage	2
parking_sessions	Vehicle occupancy intervals for a garage	1
access_events	Individual gate access decisions and plate reads	2
gate_commands	User-issued gate commands with expiry and status	1
audit_logs	User actions and optional targets/details	1
notifications	User-facing messages and read state	0

4. Entity Relationship Diagram

The diagram below summarizes the main foreign-key relationships. Solid lines represent required foreign-key references; dashed lines indicate optional references shown in the schema. The field names at the ends of the relationships are the columns that connect the records.

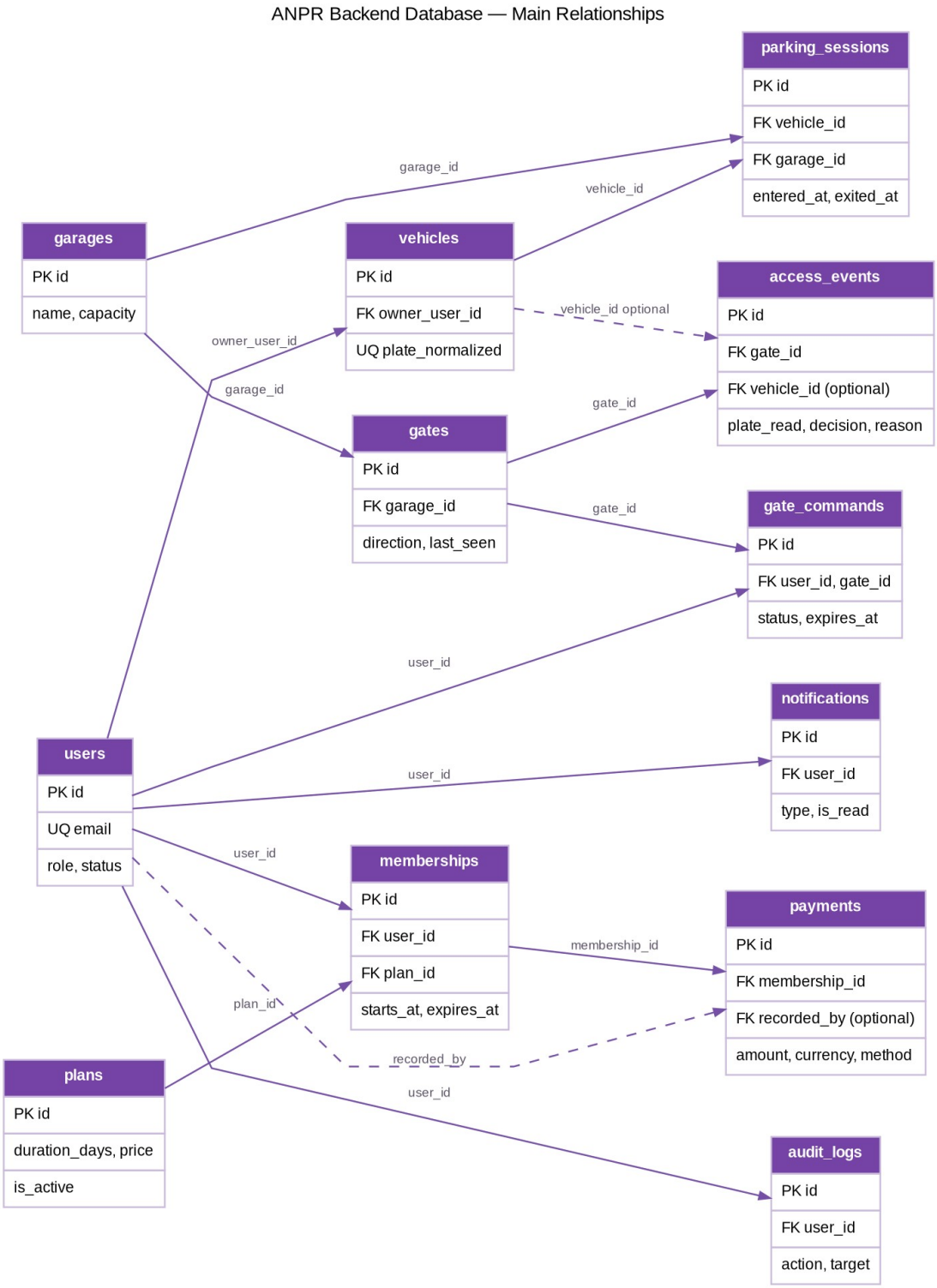


Diagram note: the relationship is defined by the foreign-key columns in schema.sql. It describes intended relational links; the application still needs to apply business rules such as active account status, eligible membership, garage capacity, and when a gate is allowed to open.

5. Relationship Model

Relationship	Interpretation
users → vehicles	One user can own multiple vehicles. Each vehicle has one required owner via vehicles.owner_user_id.
users → memberships	One user can have multiple membership records. The schema does not enforce only one current membership per user.
plans → memberships	Each membership refers to one plan. A plan may be reused by many memberships.
memberships → payments	A membership can have multiple payment records. Each payment belongs to one membership.
users → payments	payments.recorded_by optionally identifies the user who recorded the payment.
garages → gates	A garage can contain multiple gates. Each gate belongs to one garage.
vehicles / garages → parking_sessions	Each parking session records one vehicle in one garage, with entry and optional exit time.
gates → access_events	Every access event refers to a gate.
vehicles → access_events	vehicle_id is optional so a plate read can be logged even when it cannot be matched to a registered vehicle.
users / gates → gate_commands	Each command records the user issuing it and the gate it targets.
users → audit_logs	Each audit record is associated with the user responsible for the action.
users → notifications	Each notification belongs to one user.

The schema uses foreign keys to define these links, but SQLite foreign-key enforcement must be activated on each database connection before the engine will necessarily reject invalid references. This is discussed in Section 11.

6. SQL Design Conventions Used

6.1 Primary keys and AUTOINCREMENT

Every application table has an integer id primary key declared with AUTOINCREMENT. A primary key identifies one row and is the target for references from related tables. AUTOINCREMENT uses SQLite's internal sqlite_sequence metadata to track generated identifiers; that internal table should not be treated as an application entity.

6.2 NOT NULL, UNIQUE, CHECK, and DEFAULT

• NOT NULL requires a value for fields that are essential to a valid row, such as users.email or access_events.reason.

• UNIQUE prevents duplicate values in users.email and vehicles.plate_normalized. The primary-key constraints also ensure unique IDs.

• CHECK validates allowed value ranges or enumerations, such as users.role, vehicles.status, payments.amount, and expires_at > starts_at.

• DEFAULT supplies a value when an INSERT omits the column, for example active statuses and timestamps generated by datetime('now').

6.3 SQLite declared types

The schema declares TEXT, INTEGER, and REAL fields. SQLite uses type affinity rather than the same rigid column typing model as some other database engines. Application validation is still useful, particularly for date strings, plate values, currency/method fields, and the numeric confidence score.

6.4 Timestamp convention

Most created_at or occurred_at values default to SQLite datetime('now'), which yields a UTC timestamp in YYYY-MM-DD HH:MM:SS format. The starts_at, expires_at, entered_at, and paid_at fields are also TEXT. The application should store and compare them consistently, preferably in UTC with one documented format.

7. Detailed Data Dictionary

Each table section describes the declared columns and constraints from schema.sql, along with the role those fields play in the application. “Optional” means the column can be NULL according to the current schema unless another rule is noted.

7.1 users

Stores accounts, roles, and account lifecycle state.

Column	Type	Key / constraints	Meaning
id	INTEGER	Primary key; AUTOINCREMENT	Unique account identifier.
full_name	TEXT	NOT NULL	Account holder name.
email	TEXT	NOT NULL; UNIQUE	Login/contact identifier; duplicate values are rejected.
phone	TEXT	Optional	Phone/contact information; may be NULL.
password_hash	TEXT	NOT NULL	Stores a password hash, not the plaintext password. The schema does not identify the hashing algorithm.
role	TEXT	NOT NULL; DEFAULT 'member'; CHECK member/private/admin	Authorization category used by application logic.
status	TEXT	NOT NULL; DEFAULT 'active'; CHECK active/suspended/deleted	Account lifecycle state.
created_at	TEXT	NOT NULL; DEFAULT datetime('now')	Account creation timestamp in the database’s default UTC format.

Relationships / behavior: vehicles.owner_user_id references users.id; memberships.user_id, notifications.user_id, audit_logs.user_id, gate_commands.user_id and payments.recorded_by also reference users.

7.2 vehicles

Maps normalized license plates to registered vehicles and their owners.

Column	Type	Key / constraints	Meaning
id	INTEGER	Primary key; AUTOINCREMENT	Unique vehicle identifier.
owner_user_id	INTEGER	NOT NULL; FK users.id	Required owner account.
plate_normalized	TEXT	NOT NULL; UNIQUE	Canonical plate value used for lookup; the application must normalize incoming values the same way.
plate_raw	TEXT	Optional	Original or display version of the plate.
country	TEXT	Optional	Country/jurisdiction associated with the plate.
status	TEXT	NOT NULL; DEFAULT 'active'; CHECK active/blocked	Vehicle lifecycle or access status.
created_at	TEXT	NOT NULL; DEFAULT datetime('now')	Registration timestamp.

Relationships / behavior: The unique constraint is global across plate_normalized, so the same normalized plate cannot be registered twice even under different country values.

7.3 plans

Defines membership products, duration, cost, and whether the plan can be offered.

Column	Type	Key / constraints	Meaning
id	INTEGER	Primary key; AUTOINCREMENT	Unique plan identifier.
name	TEXT	NOT NULL	Display name; duplicate names are not prohibited by the schema.
duration_days	INTEGER	NOT NULL; CHECK > 0	Number of days covered by the plan.
price	INTEGER	NOT NULL; CHECK >= 0	Plan price stored as an integer.
is_active	INTEGER	NOT NULL; DEFAULT 1; CHECK 0/1	Boolean-like flag: 1 active, 0 inactive.

Relationships / behavior: memberships.plan_id references plans.id. Membership eligibility should typically check plans.is_active in addition to membership dates and status.

7.4 memberships

Records a user's membership period and its state.

Column	Type	Key / constraints	Meaning
id	INTEGER	Primary key; AUTOINCREMENT	Unique membership identifier.
user_id	INTEGER	NOT NULL; FK users.id	Member account.
plan_id	INTEGER	NOT NULL; FK plans.id	Plan to which the membership applies.
starts_at	TEXT	NOT NULL	Membership start date/time.
expires_at	TEXT	NOT NULL; CHECK expires_at > starts_at	Expiry date/time; must be after starts_at.
status	TEXT	NOT NULL; DEFAULT 'active'; CHECK active/cancelled	Membership state; dates alone do not express cancellation.
created_at	TEXT	NOT NULL; DEFAULT datetime('now')	Membership record creation timestamp.

Relationships / behavior: A user may have multiple membership rows. The schema does not enforce a unique active membership or guarantee that a user has any membership.

7.5 payments

Records payments associated with a membership.

Column	Type	Key / constraints	Meaning
id	INTEGER	Primary key; AUTOINCREMENT	Unique payment identifier.
membership_id	INTEGER	NOT NULL; FK memberships.id	Membership being paid for.
amount	INTEGER	NOT NULL; CHECK >= 0	Amount paid; zero is permitted by the current constraint.
currency	TEXT	NOT NULL; DEFAULT 'IQD'	Currency code; the schema does not restrict it to a known set.
method	TEXT	NOT NULL; DEFAULT 'cash'	Payment method label; the schema does not constrain allowed values.
paid_at	TEXT	NOT NULL; DEFAULT datetime('now')	Payment timestamp.
recorded_by	INTEGER	Optional FK users.id	User who recorded the payment, if known.

Relationships / behavior: The design permits multiple payments per membership. A complete billing workflow may need payment status, receipt/reference, refund handling, and explicit currency/method rules.

7.6 garages

Stores parking facility names and configured capacities.

Column	Type	Key / constraints	Meaning
id	INTEGER	Primary key; AUTOINCREMENT	Unique garage identifier.
name	TEXT	NOT NULL	Garage name.
capacity	INTEGER	NOT NULL; CHECK > 0	Configured capacity; must be positive.

Relationships / behavior: gates.garage_id and parking_sessions.garage_id reference garages.id. Capacity is stored here, but the database does not automatically stop sessions when capacity is reached.

7.7 gates

Defines physical or logical entry and exit gates for garages.

Column	Type	Key / constraints	Meaning
id	INTEGER	Primary key; AUTOINCREMENT	Unique gate identifier.
garage_id	INTEGER	NOT NULL; FK garages.id	Garage to which the gate belongs.
name	TEXT	NOT NULL	Gate label/name.
direction	TEXT	NOT NULL; CHECK entry/exit	Gate direction.
last_seen	TEXT	Optional	Most recent contact/status timestamp, if maintained by the application.

Relationships / behavior: access_events.gate_id and gate_commands.gate_id reference gates.id.

7.8 parking_sessions

Represents a vehicle’s stay in a garage from entry to exit.

Column	Type	Key / constraints	Meaning
id	INTEGER	Primary key; AUTOINCREMENT	Unique parking-session identifier.
vehicle_id	INTEGER	NOT NULL; FK vehicles.id	Vehicle participating in the session.
garage_id	INTEGER	NOT NULL; FK garages.id	Garage in which the session occurs.
entered_at	TEXT	NOT NULL; DEFAULT datetime('now')	Entry timestamp.
exited_at	TEXT	Optional; CHECK NULL or >= entered_at	Exit timestamp; NULL indicates an open session.

Relationships / behavior: The unique partial index one_open_session permits at most one row per vehicle with exited_at IS NULL. It does not enforce garage capacity, nor does it link sessions directly to a gate.

7.9 access_events

Logs a plate-read attempt and the decision made at a gate.

Column	Type	Key / constraints	Meaning
id	INTEGER	Primary key; AUTOINCREMENT	Unique access event identifier.
gate_id	INTEGER	NOT NULL; FK gates.id	Gate where the event occurred.
vehicle_id	INTEGER	Optional FK vehicles.id	Matched registered vehicle; NULL can represent an unmatched plate read.
plate_read	TEXT	NOT NULL	Plate string produced by the recognition/ingestion stage.

Column	Type	Key / constraints	Meaning
confidence	REAL	Optional	Recognition confidence score; the schema does not currently constrain its range.
decision	TEXT	NOT NULL; CHECK allowed/denied	Access outcome.
reason	TEXT	NOT NULL	Explanation for allowing or denying the attempt.
image_path	TEXT	Optional	Path/reference to a captured image; may be sensitive.
occurred_at	TEXT	NOT NULL; DEFAULT datetime('now')	Event timestamp.

Relationships / behavior: idx_events_time indexes occurred_at. This table is the natural destination for both accepted and rejected access decisions in the current schema.

7.10 gate_commands

Tracks commands submitted by users for gates and their lifecycle.

Column	Type	Key / constraints	Meaning
id	INTEGER	Primary key; AUTOINCREMENT	Unique command identifier.
user_id	INTEGER	NOT NULL; FK users.id	User issuing the command.
gate_id	INTEGER	NOT NULL; FK gates.id	Target gate.
status	TEXT	NOT NULL; DEFAULT 'pending'; CHECK pending/executed/expired/rejected	Command lifecycle state.
created_at	TEXT	NOT NULL; DEFAULT datetime('now')	Time the command was created.
expires_at	TEXT	NOT NULL; CHECK expires_at > created_at	Expiry timestamp; must be later than creation.

Relationships / behavior: The schema does not include an execution timestamp or command payload; those may be needed if the backend must audit exact hardware actions.

7.11 audit_logs

Keeps a history of actions taken by users.

Column	Type	Key / constraints	Meaning
id	INTEGER	Primary key; AUTOINCREMENT	Unique audit record.
user_id	INTEGER	NOT NULL; FK users.id	User responsible for the action.
action	TEXT	NOT NULL	Action name or verb.
target_type	TEXT	Optional	Type/category of affected target.
target_id	INTEGER	Optional; no FK defined	Identifier of the affected target; polymorphic references are not validated by SQLite.
details	TEXT	Optional	Additional action details.
occurred_at	TEXT	NOT NULL; DEFAULT datetime('now')	Audit timestamp.

Relationships / behavior: idx_audit_time indexes occurred_at. Avoid writing password hashes, authentication tokens, or unnecessary personal information to details.

7.12 notifications

Stores messages shown to users and whether each has been read.

Column	Type	Key / constraints	Meaning
id	INTEGER	Primary key; AUTOINCREMENT	Unique notification identifier.
user_id	INTEGER	NOT NULL; FK users.id	Recipient account.
type	TEXT	NOT NULL	Notification category; examples in the schema include expiring_soon, entered, and admin_message.
message	TEXT	NOT NULL	Human-readable notification message.
is_read	INTEGER	NOT NULL; DEFAULT 0; CHECK 0/1	Boolean-like read flag.
created_at	TEXT	NOT NULL; DEFAULT datetime('now')	Creation timestamp.

Relationships / behavior: No notification rows were present at the time of inspection. The type field is free-form text rather than a constrained enumeration.

8. How the Database Supports the ANPR Workflow

A typical backend workflow based on the current schema can be described in the following stages. These are the functions supported by the schema; the database alone does not guarantee that every stage is implemented by the current API.

• Recognition input: a computer-vision component detects a plate and may provide a normalized plate and confidence score. That component can be developed and deployed separately from the backend.

• Vehicle lookup: compare the normalized input with vehicles.plate_normalized, which is NOT NULL and UNIQUE.

• Owner and account validation: join vehicles.owner_user_id to users.id and check users.status as well as users.role.

• Membership validation: look for a membership whose status is active, whose start and expiry times cover the current instant, and whose related plan is active. Membership rows and plans are separate tables for this reason.

• Access decision: decide whether the gate should allow or deny entry. Record both outcomes in access_events, including gate_id, plate_read, decision, reason, optional vehicle_id, optional confidence, and occurred_at.

• Parking state: when the vehicle enters, create parking_sessions with entered_at. When it exits, update exited_at. The partial unique index prevents a second open session for the same vehicle.

• Supporting recordkeeping: gate_commands tracks user-originated gate commands; audit_logs captures operator actions; notifications stores messages for users; payments records billing transactions.

8.1 Typical vehicle-owner lookup

```
SELECT u.id, u.role, u.status AS user_status,
v.id AS vehicle_id, v.plate_normalized, v.status AS vehicle_status
FROM vehicles AS v
JOIN users AS u ON u.id = v.owner_user_id
WHERE v.plate_normalized = ?;
```

This explicit JOIN is easier to read than a scalar subquery and makes it straightforward to check both account and vehicle status. The parameter must already follow the same normalization convention as the stored plate.

8.2 Typical current-membership check

```
SELECT m.id, m.status, m.starts_at, m.expires_at,
p.name AS plan_name, p.is_active
FROM memberships AS m
JOIN plans AS p ON p.id = m.plan_id
WHERE m.user_id = ?
AND m.status = 'active'
AND p.is_active = 1
AND m.starts_at <= datetime('now')
AND datetime('now') <= m.expires_at
ORDER BY m.expires_at DESC
LIMIT 1;
```

This is an example, not a claim that the current API already uses this query. It assumes starts_at and expires_at are stored in the same sortable UTC text format as datetime('now'). If the application uses a different timezone convention, that convention must be handled consistently.

8.3 Typical access-event insert

```
INSERT INTO access_events
(gate_id, vehicle_id, plate_read, confidence,
decision, reason, image_path)
VALUES (?, ?, ?, ?, ?, ?, ?);
```

For denied reads or unknown plates, vehicle_id may be NULL while plate_read, decision, reason, gate_id, and event time remain available. This allows the system to preserve evidence of unsuccessful attempts instead of silently discarding them.

9. Integration Notes for the Current FastAPI Backend

9.1 Table names in the old example do not match this database

The earlier sample referenced bakery_security_logs and history, but neither appears among the 12 application tables in anpr.db. Running that SQL against this database will fail with “no such table”. Use vehicles for plate registration and access_events for access-attempt history, or update the schema intentionally if those older tables are still required.

9.2 The provided role/membership check needs additional conditions

The supplied code returns immediately for users whose role is admin or private. For other users, it reads a membership row and checks only the start and expiry timestamps. Based on the schema, the backend should decide explicitly whether to check the following fields before granting access:

• users.status: suspended or deleted users should not accidentally pass solely because their role is admin/private.

• vehicles.status: a blocked vehicle should not be treated as active merely because its owner has an eligible role or membership.

• memberships.status: a cancelled membership may have dates that still cover the current instant.

• plans.is_active: an inactive plan may need to invalidate access even if a membership record exists.

• Missing or multiple memberships: member[0] can raise an error if the user has no membership. Multiple rows exist by design, so choose the current eligible membership deterministically rather than assuming the first row is correct.

9.3 Log the decision in access_events

The schema already has a dedicated access_events table with decision constrained to allowed or denied and a required reason. A robust workflow should log both outcomes so administrators can audit access attempts. Gate identity must be supplied because gate_id is NOT NULL.

10. Indexes and Data-Integrity Rules

10.1 Explicit indexes found in the database

Index	Table	Definition / effect
idx_audit_time	audit_logs	CREATE INDEX idx_audit_time ON audit_logs (occurred_at)
idx_events_time	access_events	CREATE INDEX idx_events_time ON access_events (occurred_at)
one_open_session	parking_sessions	CREATE UNIQUE INDEX one_open_session ON parking_sessions (vehicle_id) WHERE exited_at IS NULL

The database also has automatic indexes created for the UNIQUE email and plate_normalized constraints, in addition to primary-key indexes. These support account/plate uniqueness and lookup. The two time indexes support filtering or sorting chronological events and audit logs.

10.2 Partial unique index: one_open_session

```
CREATE UNIQUE INDEX one_open_session
ON parking_sessions (vehicle_id)
WHERE exited_at IS NULL;
```

This is a partial unique index: it indexes only rows whose exited_at value is NULL. It enforces at most one open parking session per vehicle while allowing the same vehicle to have many completed sessions over time [2]. It does not enforce the garage's configured capacity.

10.3 Time-order and value checks

Rule	What it protects
plans.duration_days > 0	A plan cannot have zero or negative duration.
plans.price >= 0; payments.amount >= 0	Prices and payment amounts cannot be negative.
garages.capacity > 0	A garage must have positive configured capacity.
memberships.expires_at > starts_at	Expiry must be later than start.
parking_sessions.exited_at IS NULL OR exited_at >= entered_at	A completed session cannot exit before its entry timestamp.
gate_commands.expires_at > created_at	A command must expire after it is created.
Enumerated role/status/direction/decision values	Several critical categorical values are restricted to approved values.

10.4 Constraints not currently present

• No CHECK constraint limits access_events.confidence to a defined range such as 0.0–1.0.

• No UNIQUE constraint enforces a unique plan name or limits memberships to one active row per user.

• payments.currency and payments.method are free-form strings; application validation or lookup tables could standardize them.

• notifications.type and audit_logs.action are free-form text, which is flexible but can lead to inconsistent category names.

• audit_logs.target_id has no foreign key because target_type can represent different kinds of target; the application must validate the target if necessary.

• Garage capacity is stored but not enforced by a cross-row database constraint; backend logic or a transaction is needed to avoid overbooking under concurrent requests.

11. Current Data Snapshot and Validation Results

The following counts are aggregate-only observations from the uploaded database; they are included to show its current level of test data and not to expose individual records.

Table	Row count
users	6
vehicles	6
plans	2
memberships	5

Table	Row count
payments	2
garages	1
gates	2
parking_sessions	1
access_events	2
gate_commands	1
audit_logs	1
notifications	0

Total application rows: 29. The database contained 12 application tables, with 0 triggers. The table sqlite_sequence is SQLite-managed metadata and not included in these totals.

11.1 Selected aggregate values

Area	Observed aggregate
User roles	admin: 1, member: 4, private: 1
User statuses	active: 5, suspended: 1
Vehicle statuses	active: 5, blocked: 1
Gate directions	entry: 1, exit: 1
Access decisions	allowed: 1, denied: 1
Membership statuses	active: 5
Active-plan flags	active: 2

11.2 Integrity checks

• PRAGMA integrity_check returned: ok.

• PRAGMA foreign_key_check returned 0 violation rows.

• PRAGMA foreign_keys returned 0 on the standalone inspection connection. SQLite foreign-key enforcement is per connection, so this value does not automatically prove what the FastAPI/CS50 SQL connection uses.

12. Recommendations for the Team

Priority / recommendation	Reason and action
High priority — enable foreign-key enforcement	Enable PRAGMA foreign_keys = ON in the actual database connection used by the backend and verify the result is 1. SQLite requires this at connection scope; declaring REFERENCES clauses alone is not enough if enforcement is disabled [1].
High priority — unify date/time handling	Store timestamps consistently, preferably as UTC YYYY-MM-DD HH:MM:SS or another documented ISO-8601 format. Compare against datetime('now') or consistently converted UTC values. Avoid mixing SQLite UTC defaults with localtime values.
High priority — align the API with the current schema	Use vehicles and users for ownership lookup, memberships and plans for eligibility, access_events for decision logs, and parking_sessions for entry/exit occupancy. Remove stale references to bakery_security_logs/history unless those tables are intentionally added.
High priority — validate all eligibility states	Include user status, vehicle status, membership status, membership dates, and plan activity in the authorization decision. Handle missing membership records and multiple membership rows safely.
High priority — protect real data	Do not commit a populated production-like database to a public repository if it includes real names, emails, phone numbers, plate identifiers, password hashes, access histories, or paths to captured images. Publish a schema plus synthetic seed data for demonstrations.
Medium priority — record allowed and denied outcomes	Write one access_events row for each attempted access, with a reason, gate, decision, plate read, optional confidence, and matched vehicle when available.
Medium priority — document enumerations	Use application validation or reference tables for payment methods, currencies, notification types, and action names if consistent values matter.
Medium priority — consider lookup indexes after measuring	Foreign-key columns such as memberships.user_id, vehicles.owner_user_id, and gates.garage_id may benefit from indexes as data grows. Profile actual queries before adding indexes, because every index has a write/storage cost.
Medium priority — clarify unique plate scope	The current UNIQUE constraint applies globally to plate_normalized. If two jurisdictions can issue the same normalized plate, use a composite unique constraint on country and plate_normalized if that matches project requirements.
Medium priority — test	If the garage capacity rule is implemented, use transaction-safe logic so

Priority / recommendation	Reason and action
concurrent capacity decisions	simultaneous entry attempts cannot both consume the final space.

13. Schema-File Maintenance Notes

The uploaded schema.sql includes the statement CREATE TABLE sqlite_sequence(name,seq). This is SQLite-managed internal metadata used with AUTOINCREMENT, not a normal application table. I tested the supplied script against a fresh in-memory SQLite database: execution stops at this statement with an OperationalError stating that sqlite_sequence is reserved for internal use, after the users table has been created. Therefore, the current schema.sql cannot initialize the complete schema from scratch as written. Remove the explicit sqlite_sequence statement and rerun the script against a fresh temporary database to verify that all remaining tables and indexes are created successfully.

For reliable project setup, keep schema definitions reproducible and separate from real database contents. A common repository structure is:

```
backend/
├── app.py
├── schema.sql # DDL for the application schema
├── anpr.db # local development data; normally gitignored if sensitive
└── tests/
└── ... # tests using synthetic data
```

The actual project layout may differ. This is a recommendation, not a claim about the current repository. The database file can be generated from schema.sql and seeded with non-sensitive test records for other team members.

14. Conclusion

The current ANPR database models the core relationships needed for a license-plate access system: users own vehicles; users hold memberships that refer to plans; payments are attached to memberships; garages contain gates; parking sessions record occupancy; and access_events stores access decisions. Gate commands, audit logs, and notifications provide supporting operational functionality.

The schema already contains useful protections, including primary keys, unique normalized plates, status/value checks, time-order checks, explicit event indexes, and the partial unique index that prevents more than one open parking session for the same vehicle. The most important next steps are to enable foreign-key enforcement on the application connection, unify timestamp handling, align endpoint queries with the current table names, and make access decisions check all relevant account, vehicle, membership, and plan states.

This database report documents the backend data layer only. It does not assume that the separately developed computer-vision component is part of the same repository or runtime.

15. References

[1] SQLite — Foreign Key Support: https://www.sqlite.org/foreignkeys.html

Explains that foreign-key constraints must be enabled for each relevant database connection.

[2] SQLite — Partial Indexes: https://www.sqlite.org/partialindex.html

Explains indexes that include only rows satisfying a WHERE clause, as used by one_open_session.

[3] SQLite — CREATE TABLE: https://www.sqlite.org/lang_createtable.html

Reference for primary keys, UNIQUE, NOT NULL, CHECK, and foreign-key constraints.

[4] SQLite — Date and Time Functions: https://www.sqlite.org/lang_datefunc.html

Documents datetime(), strftime(), accepted date/time formats, and UTC behavior.

Source note: table/column names, constraints, indexes, and row counts in this report come from the uploaded schema.sql and anpr.db. Aggregate observations are snapshots and may differ as the project changes.